Line-by-Line Code Explanation: `expense_tracker.py`

This document walks through the whole program in order, from the top of the file to the bottom. Each section shows the code, then explains it line by line.

Part 1: File header and imports

```
#!/usr/bin/env python3
```

The "shebang" line. On Linux/macOS it lets you run the file directly (`./expense_tracker.py`) using whichever `python3` is installed. Windows ignores it.

```
"""
SK 2 - Expense Tracker (Financial Logic &
Analytics)
...
"""
```

A module docstring. It describes the program and shows how to run it. Python stores it but does not execute it.

```
import csv
import json
import os
import sys
from dataclasses import dataclass, asdict
from datetime import date, datetime
from decimal import Decimal,
InvalidOperation, ROUND_HALF_UP
```

- `csv` : read and write CSV files.
- `json` : read and write JSON files.
- `os` : check whether a file exists (`os.path.exists`).
- `sys` : read command-line arguments (`sys.argv`) so `--sample` works.
- `dataclass` : decorator that auto-generates `__init__` and other boilerplate for a data class.
- `asdict` : converts a dataclass object into a plain dictionary (needed for JSON).
- `date` , `datetime` : `date` is a calendar date; `datetime` is used for parsing text into dates and for the current time.
- `Decimal` : exact decimal number type, used for all money.

- `InvalidOperation`: the error `Decimal` raises when the text is not a number.
- `ROUND_HALF_UP`: rounding mode (2.675 becomes 2.68, the way people normally round money).

Part 2: Constants

```
DATA_FILE = "transactions.json"
```

The file where transactions are saved automatically.

```
CATEGORIES = ["Food", "Utilities",
"Travel", "Rent", "Shopping", "Health",
              "Entertainment", "Education",
"Salary", "Freelance", "Other"]
```

Suggested categories shown to the user when adding a transaction. The user can still type any category.

```
TYPES = ("income", "expense")
```

The only two allowed transaction types. A tuple is used because it should never change.

```
CENT = Decimal("0.01")
```

Represents "two decimal places". It is used later to round every amount to cents.

Part 3: Validation

```
class ValidationError(ValueError):
    pass
```

A custom error type. It inherits from `ValueError`. Raising `ValidationError` lets the program catch only validation problems and show friendly messages. `pass` means no extra code is needed.

`parse_date`

```
def parse_date(text):
    try:
        return
datetime.strptime(str(text).strip(), "%Y-
%m-%d").date()
    except ValueError:
        raise ValidationError("Date must be
a real date in YYYY-MM-DD format.")
```

- `str(text).strip()` : converts the input to text and removes extra spaces.

- `datetime.strptime(..., "%Y-%m-%d")`: parses text like `2025-01-15`. `%Y` = 4-digit year, `%m` = month, `%d` = day.
- It fails with `ValueError` for bad formats and impossible dates such as `2025-02-30`.
- `.date()`: keeps only the date part (drops the time).
- `except ValueError`: catches the failure and raises our own `ValidationError` with a clear message.

`parse_amount`

```
def parse_amount(text):
    try:
        amt =
Decimal(str(text).strip().replace(",", ""))
    except InvalidOperation:
        raise ValidationError("Amount must
be a number.")
```

- Removes spaces and commas, so `1,250.75` is accepted.
- `Decimal(...)` converts the text to an exact number. Text like `abc` raises `InvalidOperation`, which becomes a `ValidationError`.

```
    if not amt.is_finite():
        raise ValidationError("Amount must
be a finite number.")
```

`Decimal` accepts the words `NaN` and `Infinity`. `is_finite()` blocks them.

```
    if amt <= 0:
        raise ValidationError("Amount must
be greater than zero "
                              "
(negative/zero amounts are rejected).")
```

This is the key acceptance rule: negative and zero amounts are rejected. The direction of money is decided by `type`, not by a minus sign.

```
    return amt.quantize(CENT,
rounding=ROUND_HALF_UP)
```

Rounds to exactly 2 decimals. Example: `10.555` becomes `10.56`, and `5` becomes `5.00`.

`parse_type`

```
def parse_type(text):
    t = str(text).strip().lower()
    if t not in TYPES:
        raise ValidationError("Type must be
```

```
'income' or 'expense'.")
    return t
```

Cleans the text and lower-cases it (`Expense` becomes `expense`). If it is not in `TYPES`, it raises an error.

`parse_category`

```
def parse_category(text):
    c = str(text).strip().title()
    if not c:
        raise ValidationError("Category
cannot be empty.")
    if len(c) > 30:
        raise ValidationError("Category too
long (max 30 chars).")
    return c
```

- `.title()` makes `food` and `FOOD` both become `Food`, so filtering and grouping match consistently.
- Rejects empty text and names longer than 30 characters.

Part 4: The Transaction model

```
@dataclass
class Transaction:
```

```
    id: int
    date: str
    category: str
    amount: str      # stored as string ->
no float rounding errors
    type: str
    note: str = ""
```

- `@dataclass` automatically creates the constructor.
- Each line is a field with a type hint.
- `amount` is stored as a **string** such as `"1250.75"`. That way saving to JSON and loading back never introduces float errors.
- `note` has a default value of an empty string, so it is optional.

```
    @property
    def amt(self):
        return Decimal(self.amount)
```

A property lets you write `t.amt` (no brackets). It converts the stored string back to an exact `Decimal` whenever math is needed.

```
    @property
    def dt(self):
        return datetime.strptime(self.date,
"%Y-%m-%d").date()
```

Converts the stored date string into a real `date` object, so dates can be compared with `<` and `>`.

`build_transaction`

```
def build_transaction(tid, d, category,
amount, ttype, note=""):
    return Transaction(
        id=tid,
        date=parse_date(d).isoformat(),
        category=parse_category(category),
        amount=str(parse_amount(amount)),
        type=parse_type(ttype),
        note=str(note).strip(),
    )
```

This is the single "gate" for creating transactions. Every field goes through its validator first:

- `.isoformat()` turns a date object back into `YYYY-MM-DD` text.
- `str(parse_amount(...))` stores the rounded Decimal as text.

Because adding, loading and importing all use this function, **no invalid record can enter the system**.

Part 5: The Tracker class

Constructor

```
class Tracker:
    def __init__(self, path=DATA_FILE):
        self.path = path
        self.items = []
        self.load()
```

- `self.path` : which file to save to.
- `self.items` : the list that holds all transactions in memory.
- `self.load()` : immediately loads saved data from disk.

`_next_id`

```
    def _next_id(self):
        return max((t.id for t in
self.items), default=0) + 1
```

Finds the highest existing ID and adds 1. `default=0` handles an empty list, so the first ID is 1. Using max (not list length) keeps IDs unique even after deletions.

`add`

```
    def add(self, d, category, amount,
ttype, note=""):
```

```
        t =
build_transaction(self._next_id(), d,
category, amount, ttype, note)
        self.items.append(t)
        self.save()
        return t
```

Builds a validated transaction, adds it to the list, saves to disk, and returns it. If validation fails, an error is raised **before** anything is added.

`delete`

```
    def delete(self, tid):
        before = len(self.items)
        self.items = [t for t in self.items
if t.id != tid]
        if len(self.items) == before:
            return False
        self.save()
        return True
```

- Remembers the count before.
- Rebuilds the list without the matching ID (list comprehension).
- If the count did not change, the ID did not exist, so it returns `False`.
- Otherwise it saves and returns `True`.

`filter`

```
    def filter(self, start=None, end=None,
category=None, ttype=None):
        out = []
        cat = category.strip().title() if
category else None
```

Prepares an empty result list. The category is cleaned the same way as when saving (`Title Case`) so the comparison matches.

```
        for t in self.items:
            if start and t.dt < start:
                continue
            if end and t.dt > end:
                continue
            if cat and t.category != cat:
                continue
            if ttype and t.type != ttype:
                continue
            out.append(t)
```

For each transaction, any failed condition skips it (`continue`). A condition only applies if the filter was given (blank filters are `None`).

- `t.dt < start` skips earlier dates.
- `t.dt > end` skips later dates.
- Because only strictly earlier or later dates are skipped, **start and end dates are both included**.

```
        return sorted(out, key=lambda x:
(x.dt, x.id))
```

Returns results sorted by date, then by ID for same-day entries.

`summary` (the financial logic)

```
    @staticmethod
    def summary(txns):
```

`@staticmethod`: it does not need `self`. It works on any list given to it (all data or a filtered subset).

```
        income = sum((t.amt for t in txns
if t.type == "income"), Decimal("0"))
        expense = sum((t.amt for t in txns
if t.type == "expense"), Decimal("0"))
```

Adds up income amounts and expense amounts separately. The second argument `Decimal("0")` is the starting value, so the result stays a `Decimal` (and an empty list gives `0`, not an error).

```
        by_cat = {}
        for t in txns:
            if t.type == "expense":
                by_cat[t.category] =
```

```
by_cat.get(t.category, Decimal("0")) +
t.amt
```

Builds a dictionary of category totals. `by_cat.get(category, 0)` returns the running total so far (or 0 for a new category), then adds the current amount. Only expenses are counted.

```
        by_cat =
dict(sorted(by_cat.items(), key=lambda kv:
kv[1], reverse=True))
```

Sorts categories from highest spending to lowest.

```
        return {"income": income,
"expense": expense,
                "net": income - expense,
"by_category": by_cat}
```

Returns everything in one dictionary. Net balance = income - expense.

`save`

```
    def save(self, path=None):
        with open(path or self.path, "w",
encoding="utf-8") as f:
            json.dump([asdict(t) for t in
self.items], f, indent=2)
```

- Opens the file for writing (`"w"`).
- `asdict(t)` turns each transaction into a dictionary.
- `json.dump(..., indent=2)` writes readable, indented JSON.
- `with` closes the file automatically.

`load`

```
    def load(self):
        if not os.path.exists(self.path):
            return
```

If there is no saved file yet (first run), do nothing.

```
        try:
            with open(self.path,
encoding="utf-8") as f:
                rows = json.load(f)
            self.items =
[build_transaction(r["id"], r["date"],
r["category"],

r["amount"], r["type"], r.get("note", ""))
                           for r in rows]
```

Reads the JSON and rebuilds each transaction through `build_transaction`. So even a hand-edited file is re-validated.

```
        except (json.JSONDecodeError,
KeyError, ValidationError) as e:
            print(f"Warning: could not read
{self.path}: {e}")
            self.items = []
```

If the file is broken, has missing fields, or contains invalid values, the app prints a warning and starts empty instead of crashing.

`export_csv`

```
    def export_csv(self, txns, path):
        with open(path, "w", newline="",
encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(["id", "date",
"category", "amount", "type", "note"])
            for t in txns:
                w.writerow([t.id, t.date,
t.category, t.amount, t.type, t.note])
```

- `newline=""` prevents blank lines between rows on Windows.
- Writes a header row, then one row per transaction.
- It takes a list as input, so you can export a filtered view.

`export_json`

```
    def export_json(self, txns, path):
        with open(path, "w", encoding="utf-
8") as f:
            json.dump([asdict(t) for t in
txns], f, indent=2)
```

Same idea as `save`, but for any list and any file name.

`import_file`

```
    def import_file(self, path):
        rows = []
        if path.lower().endswith(".json"):
            with open(path, encoding="utf-
8") as f:
                rows = json.load(f)
        else:
            with open(path, newline="",
encoding="utf-8") as f:
                rows =
list(csv.DictReader(f))
```

Reads JSON if the file ends in `.json`, otherwise treats it as CSV. `DictReader` turns each CSV row into a dictionary using the header names.

```
        ok, bad = 0, []
        for i, r in enumerate(rows, 1):
            try:
```

```
self.items.append(build_transaction(
                    self._next_id(),
r["date"], r["category"],
                    r["amount"], r["type"],
r.get("note", "")))
                ok += 1
            except (ValidationError,
KeyError) as e:
                bad.append((i, str(e)))
        self.save()
        return ok, bad
```

- Counts good rows in `ok`; collects bad rows in `bad` with row number and reason.
- New IDs are assigned, so imported rows never clash with existing IDs.
- `enumerate(rows, 1)` numbers rows starting at 1.
- Saves once at the end and returns both results.

Part 6: Display helpers

```
def money(d):
    return f"{d:,.2f}"
```

Formats a number with thousands separators and 2 decimals: `1250.5` becomes `1,250.50`.

`print_table`

```
def print_table(txns):
    if not txns:
        print("  (no transactions)")
        return
```

Shows a friendly message when the list is empty.

```
    print(f"  {'ID':<4}{'Date':<12}
{'Category':<15}{'Type':<9}{'Amount':>12}
Note")
    print("  " + "-" * 70)
```

Prints the header. `<4` means left-aligned in 4 characters; `>12` means right-aligned in 12. The second line prints a divider of 70 dashes.

```
    for t in txns:
        sign = "+" if t.type == "income"
else "-"
        print(f"  {t.id:<4}{t.date:<12}
{t.category:<15}{t.type:<9}"
              f"{sign + money(t.amt):>12}
{t.note}")
```

Prints each row with the same alignment, adding `+` for income and `-` for expense in front of the amount.

`report_text`

```
def report_text(txns, title="Expense
Report"):
    s = Tracker.summary(txns)
    lines = [title, "=" * len(title),
             f"Generated :
{datetime.now():%Y-%m-%d %H:%M}",
             f"Records   : {len(txns)}",
"",
             f"Total Income   :
{money(s['income']):>14}",
             f"Total Expenses :
{money(s['expense']):>14}",
             f"Net Balance    :
{money(s['net']):>14}", "",
             "Expenses by category:"]
```

Calculates the summary, then builds the report as a list of text lines: title, underline, timestamp, record count, the three totals, and a heading.

```
    if not s["by_category"]:
        lines.append("  (none)")
    for cat, amt in
s["by_category"].items():
        pct = (amt / s["expense"] * 100) if
s["expense"] else Decimal("0")
        lines.append(f"  {cat:<15}
{money(amt):>14}  ({pct:.1f}%)")
    return "\n".join(lines)
```

- Adds "(none)" if there are no expenses.
- For each category it calculates the percentage of total expenses. The `if s["expense"] else 0` part avoids dividing by zero.
- Joins all lines into one string with line breaks.

`show_summary`

```
def show_summary(txns):
    print("\n" + report_text(txns,
"Summary") + "\n")
```

Prints the same report on screen with the title "Summary".

`plot_chart`

```
def plot_chart(txns):
    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("Matplotlib not installed.
Run: pip install matplotlib")
        return
```

The import is inside the function, so the program works even if Matplotlib is not installed. If it is missing, a hint is printed.

```
    cats = Tracker.summary(txns)
["by_category"]
    if not cats:
        print("No expenses to plot.")
        return
    plt.figure(figsize=(7, 5))
    plt.pie([float(v) for v in
cats.values()], labels=cats.keys(),
autopct="%1.1f%%")
    plt.title("Expenses by Category")
    plt.show()
```

Gets category totals, stops if there are none, then draws a pie chart. Matplotlib needs floats for drawing, so values are converted with `float(v)`. This is safe because it is only for display, not for calculation. `autopct` prints percentages on the slices.

Part 7: Input helpers

`ask`

```
def ask(prompt, parser, default=None,
optional=False):
    while True:
        raw = input(prompt).strip()
        if not raw:
            if default is not None:
```

```
                return parser(default)
            if optional:
                return None
        try:
            return parser(raw)
        except ValidationError as e:
            print(f"  ! {e}")
```

A reusable "keep asking until valid" function:

- `while True` repeats forever until it returns.
- If the user just presses Enter: use the default value if one exists; return `None` if the question is optional.
- Otherwise it runs the chosen validator (`parser`). If valid, it returns the value. If invalid, it prints the error and asks again.

`ask_filters`

```
def ask_filters():
    print("Leave blank to skip a filter.")
    start = ask("  Start date (YYYY-MM-DD):
", parse_date, optional=True)
    end = ask("  End date   (YYYY-MM-DD):
", parse_date, optional=True)
    if start and end and start > end:
        print("  ! Start is after end -
swapping.")
        start, end = end, start
```

Asks for the two dates (both optional). If the start is after the end, it swaps them automatically.

```
    cat = input("  Category: ").strip() or
None
    ttype = input("  Type (income/expense):
").strip().lower() or None
    if ttype and ttype not in TYPES:
        print("  ! Invalid type ignored.")
        ttype = None
    return start, end, cat, ttype
```

- `... or None`: a blank answer becomes `None`, meaning "no filter".
- An invalid type is ignored with a message.
- Returns four values in the same order `Tracker.filter` expects, so `tr.filter(*ask_filters())` works.

`add_flow`

```
def add_flow(tr):
    print("\nAdd transaction")
    d = ask(f"  Date [{date.today()}]: ",
parse_date, default=str(date.today()))
    ttype = ask("  Type (income/expense):
", parse_type)
    print("  Categories: " + ",
".join(CATEGORIES))
    cat = ask("  Category: ",
parse_category)
```

```
    amt = ask("  Amount: ", parse_amount)
    note = input("  Note (optional): ")
    t = tr.add(str(d), cat, amt, ttype,
note)
    print(f"  Saved: #{t.id} {t.type}
{money(t.amt)} on {t.date}")
```

Asks for each field using `ask`, so every answer is validated immediately. The date defaults to today when you press Enter. Finally it saves and prints a confirmation.

Part 8: Sample data and README generator

```
SAMPLE_ROWS = [
    ("2025-01-01", "Salary", "55000.00",
"income", "January salary"),
    ...
]
```

18 demo records (date, category, amount, type, note) covering income and many expense categories.

```
README = """# SK 2 - Expense Tracker ...
{REPORT} ..."""
```

A text template for a README. `{REPORT}` is a placeholder that is replaced with a real report later.

`make_sample`

```
def make_sample(tr):
    tr.items = []
    for r in SAMPLE_ROWS:

tr.items.append(build_transaction(tr._next_
id(), *r))
    tr.save()
```

Clears current data, then builds each sample row through the validator (`*r` unpacks the tuple into arguments). Saves to `transactions.json`.

```
    tr.export_csv(tr.items,
"sample_transactions.csv")
    tr.export_json(tr.items,
"sample_transactions.json")
    report = report_text(tr.items, "Expense
Report")
    with open("sample_report.txt", "w",
encoding="utf-8") as f:
        f.write(report)
    with open("README.md", "w",
encoding="utf-8") as f:
        f.write(README.replace("{REPORT}",
report))
    print("Created: ...")
```

Exports the sample data in both formats, writes a sample report file, and writes a `README.md` with the real report inserted at `{REPORT}`.

Part 9: The menu and `main`

```
MENU = """
=== Expense Tracker ===
 1. Add transaction
 ...
 0. Exit
"""
```

The menu text, stored as a constant.

```
def main():
    tr = Tracker()
    if "--sample" in sys.argv:
        make_sample(tr)
        return
```

- Creates the tracker (which loads saved data).
- If you ran `python expense_tracker.py --sample`, it creates the sample files and exits without showing the menu.

```
    while True:
        print(MENU)
        print(f"Balance:
```

```
{money(Tracker.summary(tr.items)['net'])}")
        choice = input("Choose: ").strip()
```

The main loop. Each time it shows the menu, the current net balance (recalculated live, which is the "real-time balance" requirement), and asks for a choice.

```
        try:
            if choice == "1":
                add_flow(tr)
            elif choice == "2":
                print_table(tr.filter())
```

- `1` runs the add flow.
- `2` calls `filter()` with no filters, which returns everything (sorted), and prints the table.

```
            elif choice == "3":
                res =
tr.filter(*ask_filters())
                print_table(res)
                show_summary(res)
```

`3` asks for filters, shows matching transactions, and shows the totals for only those transactions. `*` unpacks the four returned values into the four parameters.

```
            elif choice == "4":
                show_summary(tr.items)
            elif choice == "5":

show_summary(tr.filter(*ask_filters()))
```

`4` summarizes all data. `5` summarizes only the filtered data.

```
            elif choice == "6":
                tid = input("  Transaction
ID: ").strip()
                if tid.isdigit() and
tr.delete(int(tid)):
                    print("  Deleted.")
                else:
                    print("  ID not
found.")
```

`6` deletes by ID. `isdigit()` makes sure the input is a number before converting it with `int`.

```
            elif choice == "7":
                res =
tr.filter(*ask_filters())
                fmt = input("  Format
(csv/json/report): ").strip().lower()
                name = input("  File name
(no extension): ").strip() or "export"
```

`7` exports the filtered view. It asks for the format and a file name (default `export`).

```
                if fmt == "csv":
                    tr.export_csv(res, name
+ ".csv")
                    ...
                elif fmt == "json":
                    tr.export_json(res,
name + ".json")
                    ...
                elif fmt == "report":
                    with open(name +
".txt", "w", encoding="utf-8") as f:

f.write(report_text(res))
                    ...
                else:
                    print("  Unknown
format.")
```

Writes the chosen format, or reports an unknown format.

```
            elif choice == "8":
                p = input("  File path
(.csv/.json): ").strip()
                ok, bad = tr.import_file(p)
                print(f"  Imported {ok}
rows, skipped {len(bad)}.")
                for i, msg in bad:
```

```
                    print(f"    row {i}:
{msg}")
```

`8` imports a file, prints how many rows were imported and skipped, and lists each skipped row with its reason.

```
            elif choice == "9":

plot_chart(tr.filter(*ask_filters()))
            elif choice == "10":
                if input("  This replaces
current data. Continue? (y/n): ").lower()
== "y":
                    make_sample(tr)
```

`9` draws a pie chart for the filtered data. `10` loads sample data only after you confirm with `y`, because it replaces your current data.

```
            elif choice == "0":
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")
```

`0` exits the loop (`break`). Any other input shows "Invalid choice" and the menu appears again.

```
        except (OSError,
json.JSONDecodeError) as e:
```

```
            print(f"  ! File error: {e}")
        except (KeyboardInterrupt,
EOFError):
            print("\nGoodbye!")
            break
```

Safety nets around the whole menu:

- File problems (missing file, permission error, bad JSON) show a message instead of crashing.
- Pressing Ctrl+C or Ctrl+D exits cleanly.

```
if __name__ == "__main__":
    main()
```

Runs `main()` only when the file is executed directly. If another program imports this file, nothing runs automatically.

Summary: how the requirements are met

Requirement	Where it is handled
Record income/expense	`Transaction`, `build_transaction`, `add_flow`

Requirement	Where it is handled
with date, category, amount, type	
Real-time balance, income, category totals	`Tracker.summary`, balance shown in `main` loop
Filter by date range and category	`Tracker.filter`, `ask_filters`
Save and export CSV/JSON	`save`, `load`, `export_csv`, `export_json`
Mathematical precision	`Decimal` plus `ROUND_HALF_UP`
Accurate date interval filtering	Inclusive `<` / `>` checks in `filter`
Reject negative amounts and malformed input	`parse_amount`, `parse_date`, `parse_type`, `parse_category`
Sample dataset and README	`SAMPLE_ROWS`, `make_sample`, `README` template